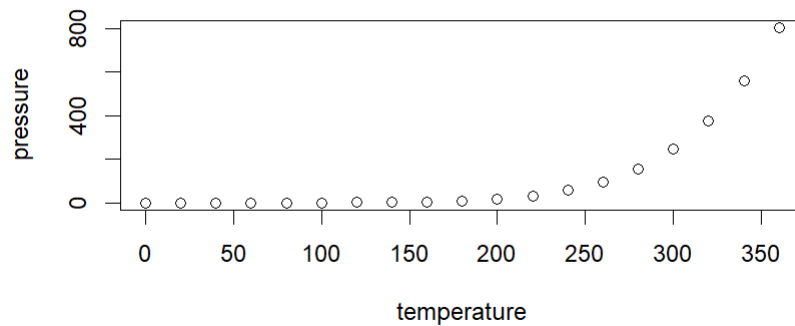


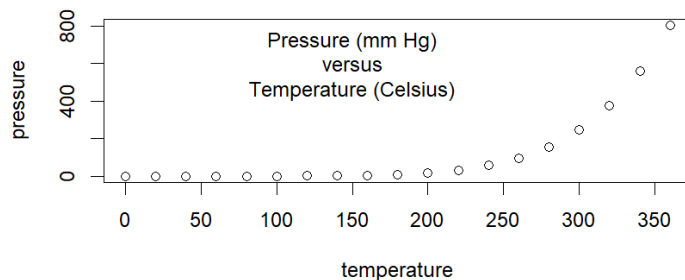
Murrell, *R Graphics* — Chapter 1 Assignment

1. Figure 1.1 — A Simple Scatterplot



```
plot(pressure)
```

A scatterplot is generated with temperature on the x-axis and pressure on the y-axis.



```
text(150, 600,
```

```
"Pressure (mm Hg)\nversus\nTemperature (Celsius)")
```

Text is added to the existing plot at coordinates. 150 is the x-coordinate where the text starts.

600 is the y-coordinate where the text starts. The `\n` character starts a new line.

2. Figure 1.2 — Some Standard Plots

```
par(mfrow = c(3, 2))
```

Divides the plotting window into 6 spaces

i. Scatterplot

```
x<- c(0.5, 2, 4, 8, 12, 16)
```

```
y1 <- c(1, 1.3, 1.9, 3.4, 3.9, 4.8)
```

```
y2 <- c(4, .8, .5, .45, .4, .3)
```

```
par(las = 1, mar = c(4, 4, 2, 4), cex = .7)
```

Values are stored for later use. Graphical parameters for label orientation, margins, and text size are set for subsequent plots.

las = 1 → Horizontal label orientation

mar → Margin sizes

cex = 0.7 → Shrinks text and symbols to 70% of their default size

```
plot.new()
```

A blank plotting region is displayed.

```
plot.window(range(x), c(0, 6))
```

The x-axis spans the range of x, and the y-axis ranges from 0 to 6.

```
lines(x, y1)
```

The first line is added to the plot.

```
lines(x, y2)
```

A second line is added to the existing axes without removing the first.

```
points(x, y1, pch = 16, cex = 2)
```

Large filled circles (pch = 16) are plotted along the first line.

```
points(x, y2, pch = 21, bg = "white", cex = 2)
```

Open circles with white fill are plotted along the second line.

```
par(col="gray50", fg="gray50", col.axis="gray50")
```

The plotting, foreground, and axis colors are set to gray50.

```
axis(1, at=seq(0, 16, 4))
```

```
axis(2, at=seq(0, 6, 2))
```

```
axis(4, at=seq(0, 6, 2))
```

Each function adds one axis: bottom, left, and right, respectively.

seq() generates a sequence. For example, seq(0, 16, 4) creates tick marks at 0, 4, 8, 12, and 16.

```
box(bty = "u")
```

A U-shaped border is drawn around the plotting region.

```
mtext("Travel Time (s)", side=1, line=2, cex=0.8)
```

```
mtext("Responses per Travel", side=2, line=2, las=0, cex=0.8)
```

```
mtext("Responses per Second", side=4, line=2, las=0, cex=0.8)
```

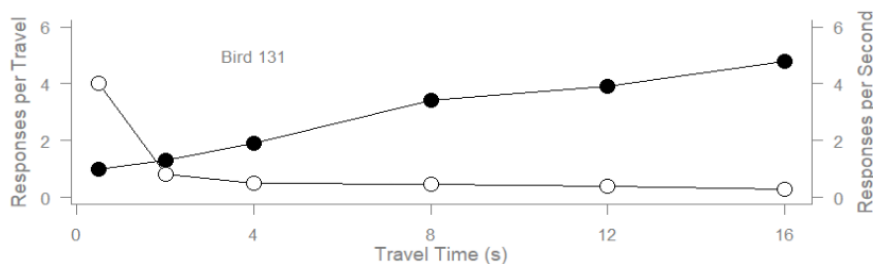
Adds labels to the axes.

```
text(4, 5, "Bird 131")
```

The label "Bird 131" is placed at coordinates (4, 5) within the plot.

```
par(mar=c(5.1, 4.1, 4.1, 2.1), col="black", fg="black", col.axis="black")
```

Sets the plot margins and makes the plot, foreground elements, and axis labels/ticks black.



ii. A Histogram with a Density Curve

```
Y <- rnorm(50)
```

50 random normal values are generated.

```
Y[Y < -3.5 | Y > 3.5] <- NA
```

It replaces values outside the range -3.5 to 3.5 with NA (missing values).

```
x <- seq(-3.5, 3.5, .1)
```

```
dn <- dnorm(x)
```

An x-grid and corresponding normal density values are computed for the reference curve.

```
par(mar=c(4.5, 4.1, 3.1, 0))
```

```
hist(Y, breaks=seq(-3.5, 3.5), ylim=c(0, 0.5),
```

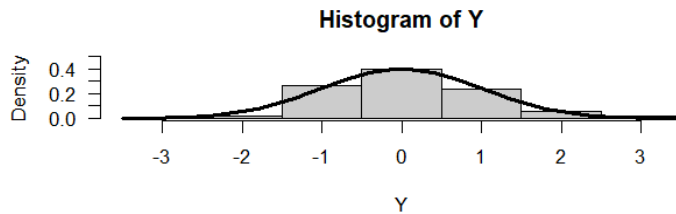
```
col="gray80", freq=FALSE)
```

Margins are set (par). A histogram appears, showing the distribution of Y.

```
lines(x, dnorm(x), lwd=2)
```

```
par(mar=c(5.1, 4.1, 4.1, 2.1))
```

A bell-curve line is added on top of the histogram bars. Margins are set (par).



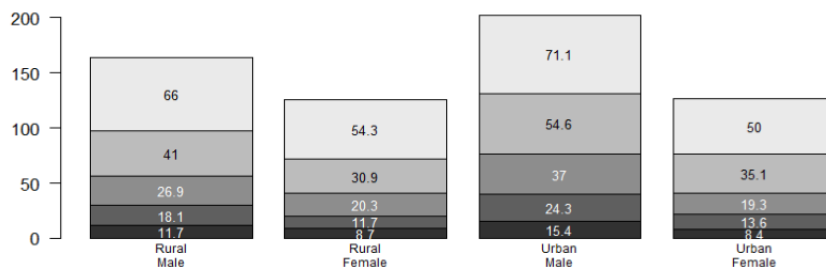
iii. A Barplot

```
par(mar=c(2, 3.1, 2, 2.1))
midpts <- barplot(VADeaths,
  col=gray(0.1 + seq(1, 9, 2)/11),
  names=rep("", 4))
```

A barplot of VADeaths appears with grayscale bars.

```
mtext(sub(" ", "\n", colnames(VADeaths)),
  at=midpts, side=1, line=0.5, cex=0.5)
text(rep(midpts, each=5), apply(VADeaths, 2, cumsum) - VADeaths/2,
  VADeaths,
  col=rep(c("white", "black"), times=3:2),
  cex=0.8)
par(mar=c(5.1, 4.1, 4.1, 2.1))
```

Custom labels are added: some in the margin (mtext), others directly on the bars (text)



iv. Boxplot

```
par(mar=c(3, 4.1, 2, 0))
boxplot(len ~ dose, data = ToothGrowth,
  boxwex = 0.25, at = 1:3 - 0.2,
  subset= supp == "VC", col="white",
  xlab="",
  ylab="tooth length", ylim=c(0,35))
mtext("Vitamin C dose (mg)", side=1, line=2.5, cex=0.8)
```

Boxplots appear, Vitamin C dose on the x-axis and tooth length on the y-axis, (boxwex = 0.25, at = 1:3 - 0.2,) narrowed and shifted slightly left.

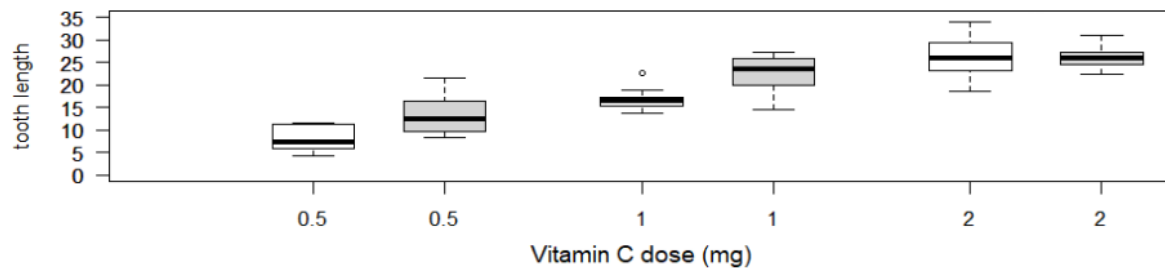
```
boxplot(len ~ dose, data = ToothGrowth, add = TRUE,
  boxwex = 0.25, at = 1:3 + 0.2,
```

```
subset= supp == "OJ")
```

Adds three narrow boxplots for the Orange Juice (OJ) supplement to an existing boxplot, positioned slightly to the right of the original boxes.

```
legend(1.5, 9, c("Ascorbic acid", "Orange juice"),
      fill = c("white", "gray"),
      bty="n")
par(mar=c(5.1, 4.1, 4.1, 2.1))
```

Adds legend and labels.

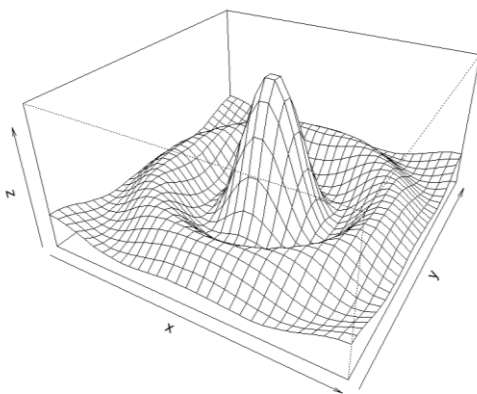


v. 3D Perspective Plot

```
x <- seq(-10, 10, length= 30)
y <- x
f <- function(x,y) { r <- sqrt(x^2+y^2); 10 * sin(r)/r }
z <- outer(x, y, f)
z[is.na(z)] <- 1
# 0.5 to include z axis label
par(mar=c(0, 0.5, 0, 0), lwd=0.5)
persp(x, y, z, theta = 30, phi = 30,

      expand = 0.5)
par(mar=c(5.1, 4.1, 4.1, 2.1), lwd=1)
```

A 3D surface plot appears, theta - horizontal (azimuthal) viewing angle, phi - vertical viewing angle, expand - vertical scaling of the surface



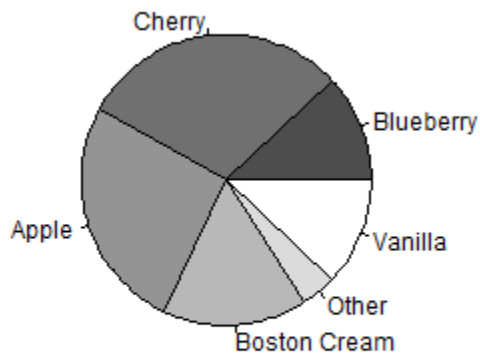
vi. Pie Chart

```
par(mar=c(0, 2, 1, 2), xpd=FALSE, cex=0.5)
pie.sales <- c(0.12, 0.3, 0.26, 0.16, 0.04, 0.12)
names(pie.sales) <- c("Blueberry", "Cherry",
  "Apple", "Boston Cream", "Other", "Vanilla")
```

The 6 values (in proportions) and labels attached to the values are stored.

```
pie(pie.sales, col = gray(seq(0.3,1.0,length=6)))
```

A pie chart with six labeled, grayscale-shaded slices appears.



3. Figure 1.3 — Customized Scatterplot

I have divided the code in 4 sections to understand it better.

Section 1:

```
params <- function(N, breaks, p=seq(0.001, 1, length=100)) {
  list(N=N, T=1/breaks, p=p, q=1-p)
}
```

```
pdfcomp <- function(comp, params) {
  n <- params$T
  p <- params$p
  q <- params$q
  y <- round(comp/n)
  choose(n, comp)*p^comp*q^(n-comp) / (1 - q^n)
}
```

```

# Expected num sherds (for a vessel) [=completeness]
expcomp <- function(params) {
  params$T*params$p/(1-params$q^params$T)
}

# Variance of num sherds (for a vessel)
varcomp <- function(params) {
  n <- params$T
  p <- params$p
  q <- params$q
  # From Johnson & Kotz
  (n*p*q / (1 - q^n)) - (n^2*p^2*q^n / (1 - q^n)^2)
  # n^2 times Thomas Yee's formula
  # n^2*((p*(1 + p*(n - 1)) / (n*(1 - q^n))) - (p^2 / (1 - q^n)^2))
}

# Expected value of completeness (for a sample of vessels)
expmeancomp <- function(params) {
  expcomp(params)
}

# Variance of completeness (for a sample of vessels)
# Use the expected number of vessels in sample as denominator
varmeancomp <- function(params) {
  varcomp(params)/(numvess(params))
}

numvess <- function(params) {
  params$N*(1-params$q^params$T)
}

```

The first section defines functions that prepare parameters and perform statistical calculations. params() stores values such as the number of vessels (N), brokenness (breaks), sampling fraction (p), and $q = 1-p$. Other functions calculate expected completeness (expcomp()), variance (varcomp()), expected mean completeness, and the expected number of vessels (numvess()).

Section 2:

```
ecomp <- function(p, T, comp) {
  q <- 1 - p
  T*p/(1 - q^T) - comp
}
```

```
estN <- function(comp, broke, n) {
  T <- 1/broke
  n / (1 - (1 - uniroot(ecomp, c(0.00001, 1), T=T, comp=comp)$root)^T)
}
```

The next group of functions is used to estimate important values. ecomp() calculates the difference between expected and observed completeness, while estN() uses uniroot() to estimate the total number of vessels. lowerCI() and upperCI() calculate the lower and upper limits around the expected completeness. critP() finds critical sampling-fraction values that are later used to identify important areas on the graph.

Section 3:

```
nvessscale <- function(params, xlim, ylim, new=TRUE) {
  if (new)
    par(new=TRUE)
  plot(0:1, c(1, params$N), type="n", axes=!new, ann=FALSE,
       xlim=xlim, ylim=ylim)
}
```

```
compscale <- function(params, xlim, ylim, new=TRUE) {
  if (new)
    par(new=TRUE)
```

```

plot(0:1, c(1, params$T), type="n", axes=!new, ann=FALSE,
     xlim=xlim, ylim=ylim)
}

```

```

lowerCI <- function(p, N, breaks, lb) {
  params <- params(N, breaks, p)
  expmeancomp(params) - 2*sqrt(varmeancomp(params)) - lb
}

```

```

upperCI <- function(p, N, breaks, lb) {
  params <- params(N, breaks, p)
  expmeancomp(params) + 2*sqrt(varmeancomp(params)) - lb
}

```

```

critP <- function(comp, params) {
  c(uniroot(lowerCI, c(0.00001, 1), N=params$N,
    breaks=1/params$T, lb=max(comp))$root,
    if (upperCI(0.00001, params$N, 1/params$T, min(comp)) > 0) 0
    else uniroot(upperCI, c(0.00001, 1), N=params$N,
    breaks=1/params$T, lb=min(comp))$root)
}

```

```

anncomp <- function(params, comp, xlim, ylim, cylim) {
  cp <- critP(comp, params)
  nv <- numvess(params(params$N, 1/params$T, cp))
  nvessscale(params, xlim, ylim)
  polygon(c(cp[2], cp[2], 0, 0, cp[1], cp[1]),
    c(0, nv[2], nv[2], nv[1], nv[1], 0),
    col="gray90", border=NA)
  text(0, nv[1], paste(round(nv[1]),
    " (", round(100*nv[1]/params$N), "%)", sep=""),

```

```

    adj=c(0, 0), col="gray")
text(0, nv[2], paste(round(nv[2]),
    " (", round(100*nv[2]/params$N), "%)", sep=""),
    adj=c(0, 1), col="gray")
compscale(params, xlim, ylim)
segments(1, min(comp), cp[2], comp, col="gray")
segments(1, max(comp), cp[1], comp, col="gray")
text(1, comp, paste(comp, collapse="-"), adj=c(1, 0), col="gray")
}

```

The plotting functions then build the customized graph. `nvessscale()` creates the scale for the number of vessels, while `compscale()` creates the completeness scale. `anncomp()` adds graphical elements such as the gray shaded region, labels, and connecting lines. The main function, `plotPars()`, brings these pieces together and draws the expected completeness line, upper and lower limits, number-of-vessels line, shaded region, axis labels, and title.

Section 4:

```

plotPars <- function(params, comp, xlim=NULL, ylim=NULL) {
  mean <- expmeancomp(params)
  var <- 2*sqrt(varmeancomp(params))
  lb <- mean - var
  ub <- mean + var
  par(mar=c(5, 4, 4, 4))
  if (is.null(ylim))
    ylim <- ylim
  else
    ylim <- c(1 + ((ylim[1] - 1)/(params$N - 1))*(params$T - 1),
      1 + ((ylim[2] - 1)/(params$N - 1))*(params$T - 1))
  nvessscale(params, xlim, ylim, new=FALSE)
  compscale(params, xlim, ylim)
  polygon(c(params$p, rev(params$p)), c(lb, rev(ub)),
    col="gray90", border=NA)
}

```

```

anncomp(params, comp, xlim, ylim, cylim)
nvessscale(params, xlim, ylim)
mtext("Number of Vessels", side=2, line=3)
mtext("Sampling Fraction", side=1, line=3)
lines(params$p, numvess(params))
par(new=TRUE)
compscale(params, xlim, cylim)
mtext("Completeness", side=4, line=3)
axis(4)
lines(params$p, mean, lty="dashed")
lines(params$p, lb, lty="dotted")
lines(params$p, ub, lty="dotted")
mtext(paste("N = ", round(params$N),
            "   brokenness = ", round(1/params$T, 3), sep=""),
      side=3, line=2)
}

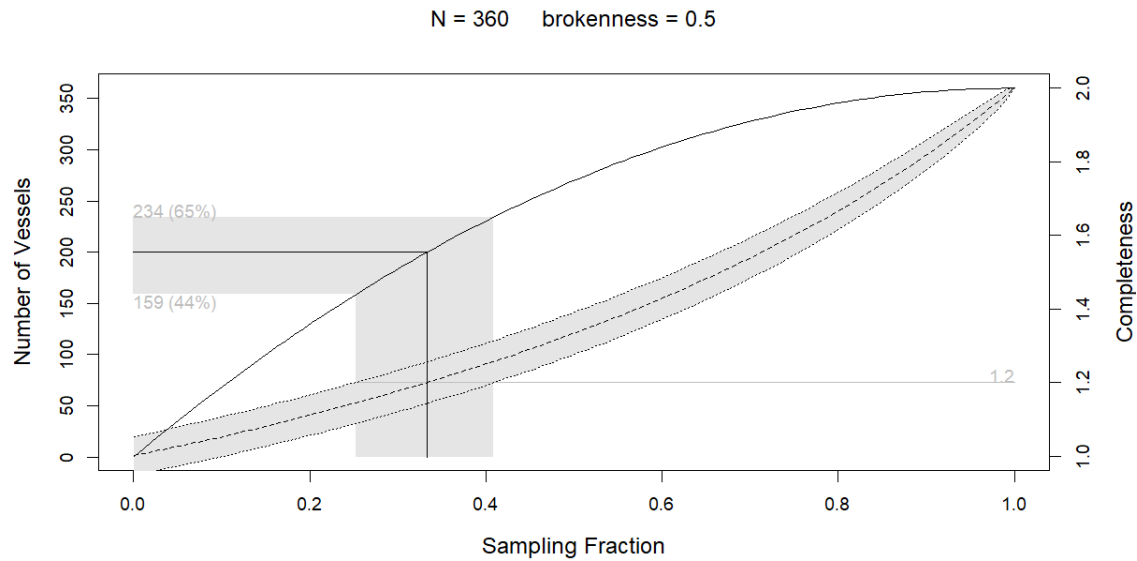
```

```

par(cex=0.8, mar=c(3, 3, 3, 3))
p6 <- params(estN(1.2, 0.5, 200), 0.5)
plotPars(p6, 1.2)
nvessscale(p6, NULL, NULL)
pcrit <- 1 - (1 - 200/estN(1.2, 0.5, 200))^(1/p6$T)
lines(c(0, pcrit), c(200, 200))
lines(c(pcrit, pcrit), c(200, 0))

```

Finally, the last commands run the functions and produce the actual graph. `p6 <- params(estN(1.2, 0.5, 200), 0.5)` estimates the number of vessels and creates the parameter list. `plotPars(p6, 1.2)` generates the main customized plot. `pcrit` identifies the critical sampling fraction, and the two `lines()` commands draw horizontal and vertical reference lines to show where that value intersects the 200-vessel level.



4. **Figure 1.4 — Dramatized Barplot (tiger populations)**

Error in link (<http://www.globaltiger.org/population.htm>)

5. **Figure 1.5 — Trellis Dotplot (lattice)**

```
library(lattice)
```

```
trellis.par.set(list(fontsize=list(text=6),
                     par.xlab.text=list(cex=1.5),
                     add.text=list(cex=1.5),
                     superpose.symbol=list(cex=.5)))
```

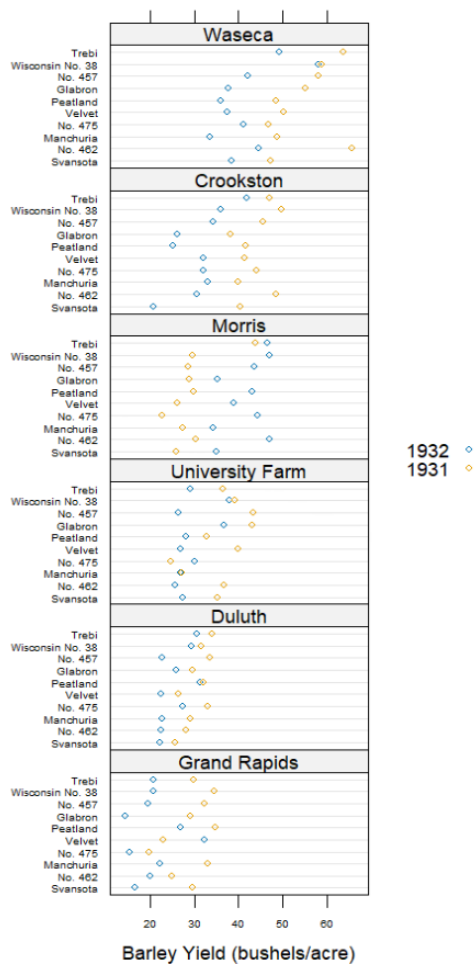
```
key <- simpleKey(levels(lattice::barley$year), space = "right")
```

```
key$text$cex <- 1.5
```

```
print(
```

```
  dotplot(variety ~ yield | site, data = lattice::barley, groups = year,
           key = key,
           xlab = "Barley Yield (bushels/acre) ",
           aspect=0.5, layout = c(1,6), ylab=NULL)
```

A *multi-panel* dotplot appears — one panel per site, each showing variety vs. yield, colored/grouped by year.



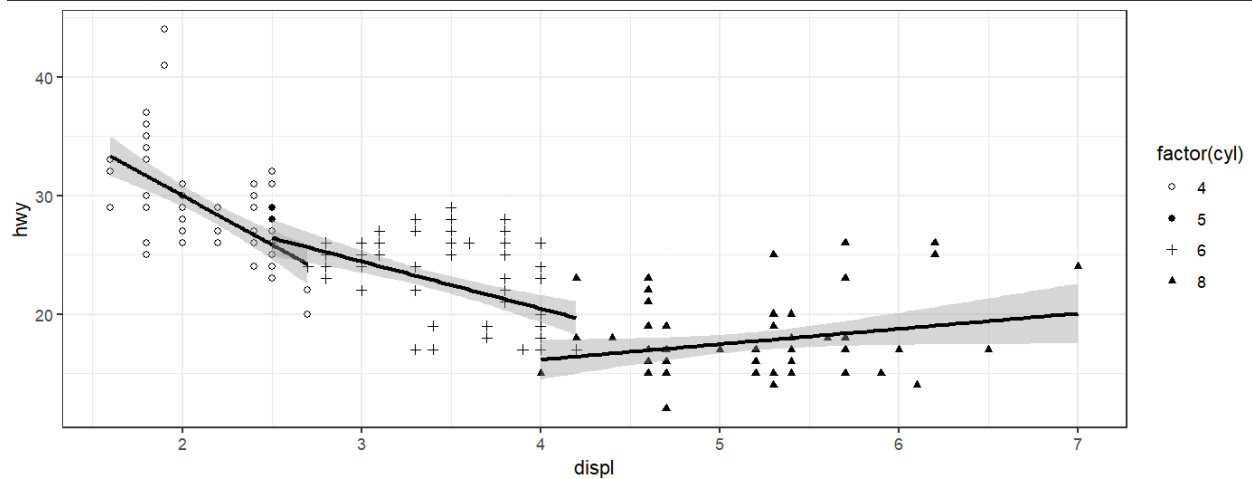
6. **Figure 1.6 — ggplot2 Scatterplot**

```
library(ggplot2)

library(ggplot2)

print(
  ggplot(data=ggplot2::mpg, aes(x=displ, y=hwy, shape=factor(cyl))) +
    geom_point() +
    stat_smooth(method="lm", colour="black") +
    scale_shape_manual(values=c(1, 16, 3, 17)) +
    theme_bw()
)
```

This generates a complete scatterplot with data points, a fitted linear trend line, 5 point shapes based on the number of cylinders, and a black-and-white theme.



7. **Figure 1.7 — Map of New Zealand**

Install packages : `install.packages(c("maps", "mapdata")), install.packages("mapproj")`

```
library(maps)
```

```
library(mapdata)
```

```
par(mar=rep(1, 4))
```

```
maps::map("nzHires", fill=TRUE, col="gray80",  
  regions=c("North Island", "South Island", "Stewart Island"))
```

Map appears.

```
points(174.75, -36.87, pch=16, cex=2,
```

```
  col=rgb(0,0,0,.5))
```

```
arrows(172, -36.87, 174, -36.87, lwd=3)
```

```
text(172, -36.87, "Auckland ", adj=1, cex=2)
```

Label Auckland appears

```
# mini world map as guide
```

```
maplocs <- maps::map(projection="sp_mercator", wrap=TRUE, lwd=0.1,
```

```
  col="gray", ylim=c(-60, 75),
```

```

interior=FALSE, orientation=c(90, 180, 0), add=TRUE,
plot=FALSE)
xrange <- range(maplocs$x, na.rm=TRUE)
yrange <- range(maplocs$y, na.rm=TRUE)
aspect <- abs(diff(yrange))/abs(diff(xrange))
# customised to 6.5 by 4.5 figure size
par(fig=c(0.99 - 0.5, 0.99, 0.01, 0.01 + 0.5*aspect*4.5/6.5),
    mar=rep(0, 4), new=TRUE)
plot.new()
plot.window(xlim=xrange,
            ylim=yrange)

```

Working till here

```

maps::map(projection="sp_mercator", wrap=TRUE, lwd=0.5, ylim=c(-60, 75),
interior=FALSE, orientation=c(90, 180, 0), add=TRUE)
symbols(-.13, -0.8, circles=1, inches=0.1, add=TRUE, bg=rgb(0,0,0,.2))
box()

```

Getting Warning message:

In maps::map(projection = "sp_mercator", wrap = TRUE, lwd = 0.5, :
projection failed for some data

8. Figure 1.8 — A plot of financial data

Install Packages: install.packages("quantmod")

```

library(quantmod)
getSymbols("AABA", src = "yahoo")
chartSeries(get("AABA"), subset = "last 4 months")

```

Getting error: Failed to open

'https://query2.finance.yahoo.com/v8/finance/chart/AABA?period1=1167609600&period2=1790121600
&interval=1d': The requested URL returned error: 404

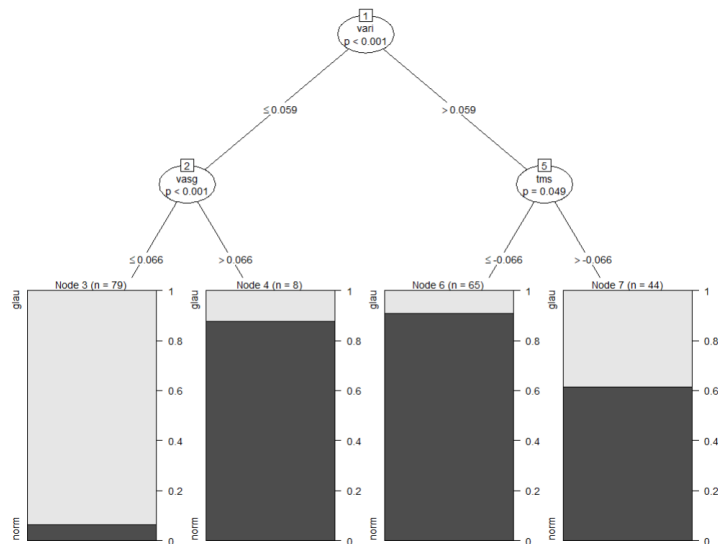
9. Figure 1.9 - Decision Tree

```
install.packages("ipred"), install.packages("TH.data"),
install.packages("party")library(party); library(ipred)
```

```
data("GlaucomaM", package = "TH.data")
```

```
fm.class <- party::ctree(Class ~ ., data = glau)
```

```
plot(fm.class, new = FALSE, terminal.panel = myNode)
```



10. Figure 1.10 — A table-like plot

- i. Prepare the data, store labels, case numbers, odds ratios, confidence intervals, row positions, and box sizes in col1, col2, and col3.

```
col1 <- list(labels=
```

```
list(textGrob("Centre", x=0, just="left",
```

```
gp=gpar(fontface="bold", col="white")),
```

```
textGrob("Thailand", x=0, just="left"),
```

```
textGrob("Philippines", x=0, just="left"),
```

```
textGrob("All in situ", x=0, just="left",
```

```
gp=gpar(fontface="bold.italic")),
```

```
textGrob("Colombia", x=0, just="left"),
```

```
textGrob("Spain", x=0, just="left"),
```

```
textGrob("All invasive", x=0, just="left",
```

```
gp=gpar(fontface="bold.italic")),
```

```

      textGrob("All", x=0, just="left",
              gp=gpar(fontface="bold"))),
  rows=c(1, 5, 6, 8, 11, 12, 14, 16))

# Labels in col 1 which are not used to calculate the
# column width (they spill over into col 2)
col1plus <- list(labels=
  list(textGrob("Carcinoma in situ", x=0, just="left",
              gp=gpar(fontface="bold.italic")),
        textGrob("Invasive cancer", x=0, just="left",
              gp=gpar(fontface="bold.italic"))),
  rows=c(4, 10))

# Data for column 2
col2 <- list(labels=
  list(textGrob("Cases", x=1, just="right",
              gp=gpar(fontface="bold", col="white")),
        textGrob("327", x=1, just="right"),
        textGrob("319", x=1, just="right"),
        textGrob("1462", x=1, just="right",
              gp=gpar(fontface="bold")),
        textGrob("96", x=1, just="right"),
        textGrob("115", x=1, just="right"),
        textGrob("211", x=1, just="right",
              gp=gpar(fontface="bold")),
        textGrob("1673", x=1, just="right",
              gp=gpar(fontface="bold"))),
  rows=c(1, 5, 6, 8, 11, 12, 14, 16))

# Data for column 3 (width specified as a physical size below)

```

```
col3 <- list(OR=c(0.72, 1.27, 1.17, 2.97, 1.86, 2.01, 1.20),
            LL=c(0.52, 0.87, 1.03, 1.42, 0.46, 1.09, 1.07),
            UL=c(1.00, 1.85, 1.32, 6.21, 7.51, 3.71, 1.35),
            rows=c(5, 6, 8, 11, 12, 14, 16),
            # "s" means summary, "n" means normal
            type=c("n", "n", "s", "n", "n", "s", "s"))
```

```
# Sizes of boxes
```

```
information <- sqrt(1 / ((log(col3$UL) - log(col3$OR))/1.96))
col3$sizes <- information/max(information)
```

```
# Width of column 3
```

```
col3width <- unit(1.5, "inches")
```

```
# Range on the x-axis for column 3
```

```
col3$range <- c(0, 4)
```

ii. Create drawing functions - Define functions to draw the labels, squares with confidence intervals, summary diamonds, axes, and forest-plot elements.

```
# Function to draw a cell in a text column
```

```
drawLabelCol <- function(col, j) {
  for (i in 1:length(col$rows)) {
    pushViewport(viewport(layout.pos.row=col$rows[i], layout.pos.col=j))
    # Labels are grobs containing their location so just
    # have to grid.draw() them
    grid.draw(col$labels[[i]])
    popViewport()
  }
}
```

```
# Function to draw a non-summary rect-plus-CI
```

```

drawNormalCI <- function(LL, OR, UL, size) {
  # NOTE the use of "native" units to position relative to
  # the x-axis scale, and "snpc" units to size relative to
  # the height of the row
  # ("snpc" stands for "square normalised parent coordinates"
  # which means that the value is calculated as a proportion
  # of the width and height of the current viewport and the
  # physically smaller of these is used)
  grid.rect(x=unit(OR, "native"),
            width=unit(size, "snpc"), height=unit(size, "snpc"),
            gp=gpar(fill="black"))
  # Draw arrow if exceed col range
  # convertX() used to convert between coordinate systems
  if (convertX(unit(UL, "native"), "npc", valueOnly=TRUE) > 1)
    grid.lines(x=unit(c(LL, 1), c("native", "npc")), y=.5,
              arrow=arrow(length=unit(0.05, "inches")))
  else {
    # Draw line white if totally inside rect
    lineCol <- if ((convertX(unit(OR, "native") + unit(0.5*size, "lines"),
                          "native", valueOnly=TRUE) > UL) &&
                  (convertX(unit(OR, "native") - unit(0.5*size, "lines"),
                          "native", valueOnly=TRUE) < LL))
      "white"
    else
      "black"
    grid.lines(x=unit(c(LL, UL), "native"), y=0.5,
              gp=gpar(col=lineCol))
  }
}

```

```

# Function to draw a summary "diamond"
drawSummaryCI <- function(LL, OR, UL, size) {
  # Not sure how to calc the heights of the diamonds so
  # I'm just using half the height of the equivalent rect
  grid.polygon(x=unit(c(LL, OR, UL, OR), "native"),
    y=unit(0.5 + c(0, 0.25*size, 0, -0.25*size), "npc"))
}

# Function to draw a "data" column
drawDataCol <- function(col, j) {
  pushViewport(viewport(layout.pos.col=j, xscale=col$range))
  grid.lines(x=unit(1, "native"), y=0:1)
  # Assume that last value in col is "All"
  grid.lines(x=unit(col$OR[length(col$OR)], "native"),
    y=0:1, gp=gpar(lty="dashed"))
  grid.xaxis(gp=gpar(cex=0.6))
  grid.text("OR", y=unit(-2, "lines"))
  popViewport()
  for (i in 1:length(col$rows)) {
    pushViewport(viewport(layout.pos.row=col$rows[i], layout.pos.col=j,
      xscale=col$range))
    if (col$type[i] == "n")
      drawNormalCI(col$LL[i], col$OR[i], col$UL[i], col$sizes[i])
    else
      drawSummaryCI(col$LL[i], col$OR[i], col$UL[i], col$sizes[i])
    popViewport()
  }
}

```

iii. Draw the final figure - Create the 16×5 grid layout, add the black/gray backgrounds, draw the labels and case numbers, and finally draw the forest plot.

```
# Draw the table
#
# The table is just a big layout
#
# All rows are the height of 1 line of text
#
# Widths of column 1 and 2 are based on widths of labels in
# col$labels and col2$labels
pushViewport(viewport(layout=grid.layout(16, 5,
    widths=
    unit.c(max(unit(rep(1, 8), "grobwidth", col1$labels)),
    colgap,
    max(unit(rep(1, 8), "grobwidth", col2$labels)),
    colgap,
    col3width),
    heights=unit(c(1, 0, rep(1, 14)), "lines"))))
pushViewport(viewport(layout.pos.row=1))
grid.rect(gp=gpar(col=NA, fill="black"))
popViewport()
for (i in c(8, 14, 16)) {
  pushViewport(viewport(layout.pos.row=i))
  grid.rect(gp=gpar(col=NA, fill="gray80"))
  popViewport()
}
drawLabelCol(col1, 1)
drawLabelCol(col1plus, 1)
drawLabelCol(col2, 3)
drawDataCol(col3, 5)
```

popViewport()

